Informe Laboratorio 2

Sección 1

Alumno Oseas Poveda Huerta
e-mail: oseas.poveda@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción de actividades **2**

2. Desarrollo de actividades según criterio de rúbrica **2**
2.1. Levantamiento de docker para correr DVWA (dvwa) 2
2.2. Redirección de puertos en docker (dvwa) 3
2.3. Obtención de consulta a replicar (burp) 3
2.4. Identificación de campos a modificar (burp) 5
2.5. Obtención de diccionarios para el ataque (burp) 5
2.6. Obtención de al menos 2 pares (burp) 7
2.7. Obtención de código de inspect element (curl) 9
2.8. Utilización de curl por terminal (curl) 10
2.9. Demuestra 4 diferencias (curl) 11
2.10. Instalación y versión a utilizar (hydra) 12
2.11. Explicación de comando a utilizar (hydra) 12
2.12. Obtención de al menos 2 pares (hydra) 12
2.13. Explicación paquete curl (tráfico) 13
2.14. Explicación paquete burp (tráfico) 15
2.15. Explicación paquete hydra (tráfico) 16
2.16. Mención de las diferencias (tráfico) 17
2.17. Detección de SW (tráfico) 18

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA
(Damn Vulnerable Web App - https://github.com/digininja/DVWA (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (https://portswigger.net/burp/communitydownload (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para empezar el laboratorio primero se deben de levantar un entorno en docker con la imagen que se nos indica en la tarea, la cual es una imagen de `vulnerables/web-dvwa` con el siguiente comando en la termianl de kali:

```
docker pull vulnerables/web-dvwa
docker run -d --name dvwa -p 8080:80 vulnerables/web-dvwa
```

- `-d`: ejecuta el contenedor en segundo plano (*detached*).
- `--name dvwa`: asigna un nombre al contenedor para administrarlo fácilmente (`docker stop/start dvwa`).
- `-p 8080:80`: redirige el puerto 8080 del host hacia el puerto 80 del contenedor (sera la dirección a la cual se accede tanto para el wireshark como para el link de la pagina de vulnerabilidades).



Figura 1: Contenedor Docker en ejecución, comando utilizado y DVWA funcionando en el navegador

Es importante mencionar lo de que se activa en segundo plano porque a mitad del laboratorio se me cerro la maquina virtual y tuve que reiniciar todo, y al eliminar el docker tuve que eliminar todo lo activo e inactivo.

2.2. Redirección de puertos en docker (dvwa)

Como se observa en la Figura 1, se puede evidenciar que se activo el servidor por el cual uno ingresara a la pagina de vulnerabilidades, y como esta escrito en una de las descripciones el puerto nos permite entrar en la siguietne url `http://localhost:8080` la cual es DVWA donde se obrara todo el ataque.

2.3. Obtención de consulta a replicar (burp)

Se utilizó Burp Suite Community Edition para interceptar el tráfico HTTP generado al autenticarse en el formulario `/vulnerabilities/brute/`. Con la opción *Intercept is on* habilitada en el proxy, se ingresaron usuarios y contraseñas aleatorias en este caso se utilizo test y test .

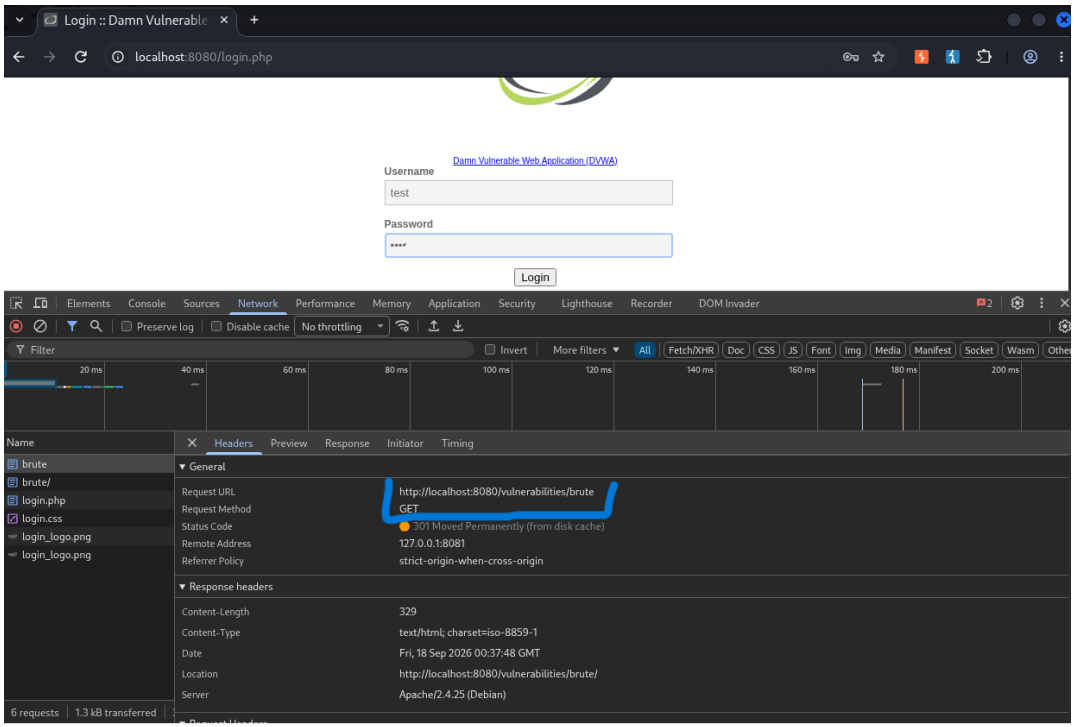


Figura 2: Pestaña Proxy de Burp Suite con *Intercept is on*, interceptando la petición

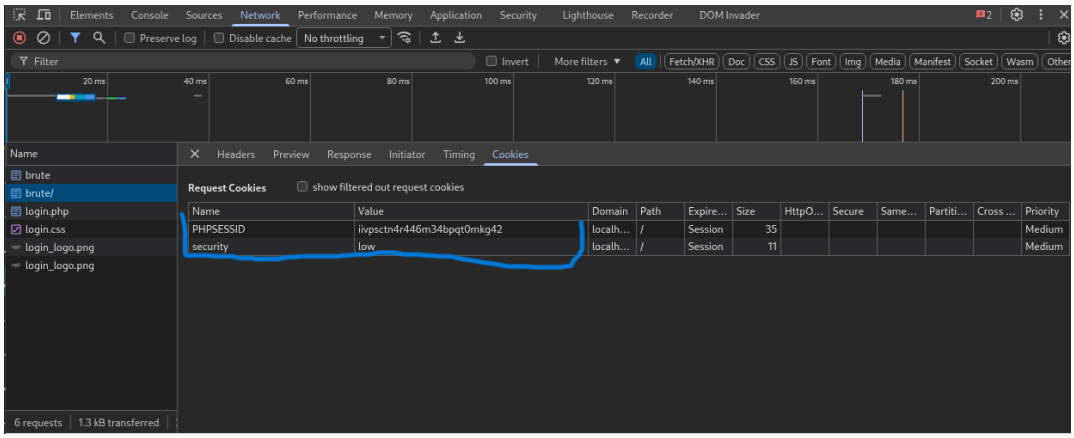


Figura 3: Pestaña Proxy de Burp Suite con *Intercept is on*, interceptando la petición

La captura evidencia la solicitud emitida por el cliente:

```
GET /vulnerabilities/brute/?username=test&password=test&Login=Login HTTP/1.1
Host: localhost:8080
Cookie: PHPSESSID=<tu_sesion>; security=low
```

En la figura3 se evidencia el método GET como también se evidencia el identificador de la sesión que se llama (`PHPSESSID`), con esto ya se puede realizar todo lo que

2.4. Identificación de campos a modificar (burp)

Los campos a modificar para lograr ingresar se los enviamos a la opción de Intruder, donde se define ussername y password para lograr ingresar

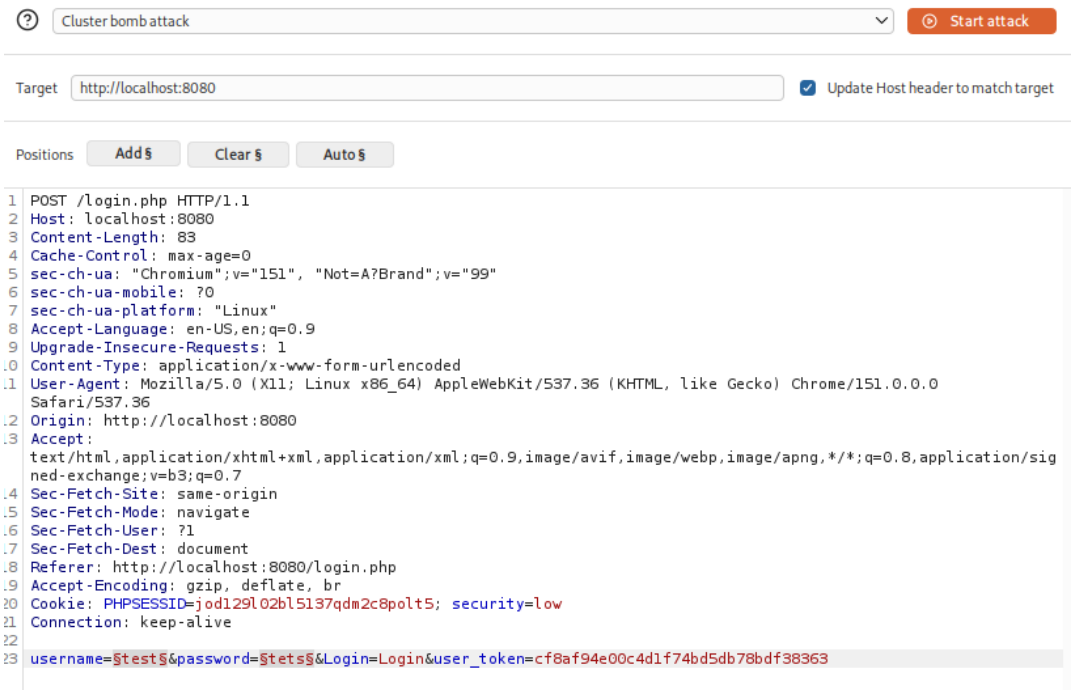


Figura 4: ussername y password seleccionados para su modificacion

Se seleccionó el tipo de ataque **Cluster Bomb**, diseñado para iterar múltiples diccionarios probando todas las combinaciones posibles entre ambas variables.

2.5. Obtención de diccionarios para el ataque (burp)

En la pestaña *Payloads* se cargó un diccionario distinto para cada posición: el *Payload set 1* con la lista de usuarios y el *Payload set 2* con la lista de contraseñas.

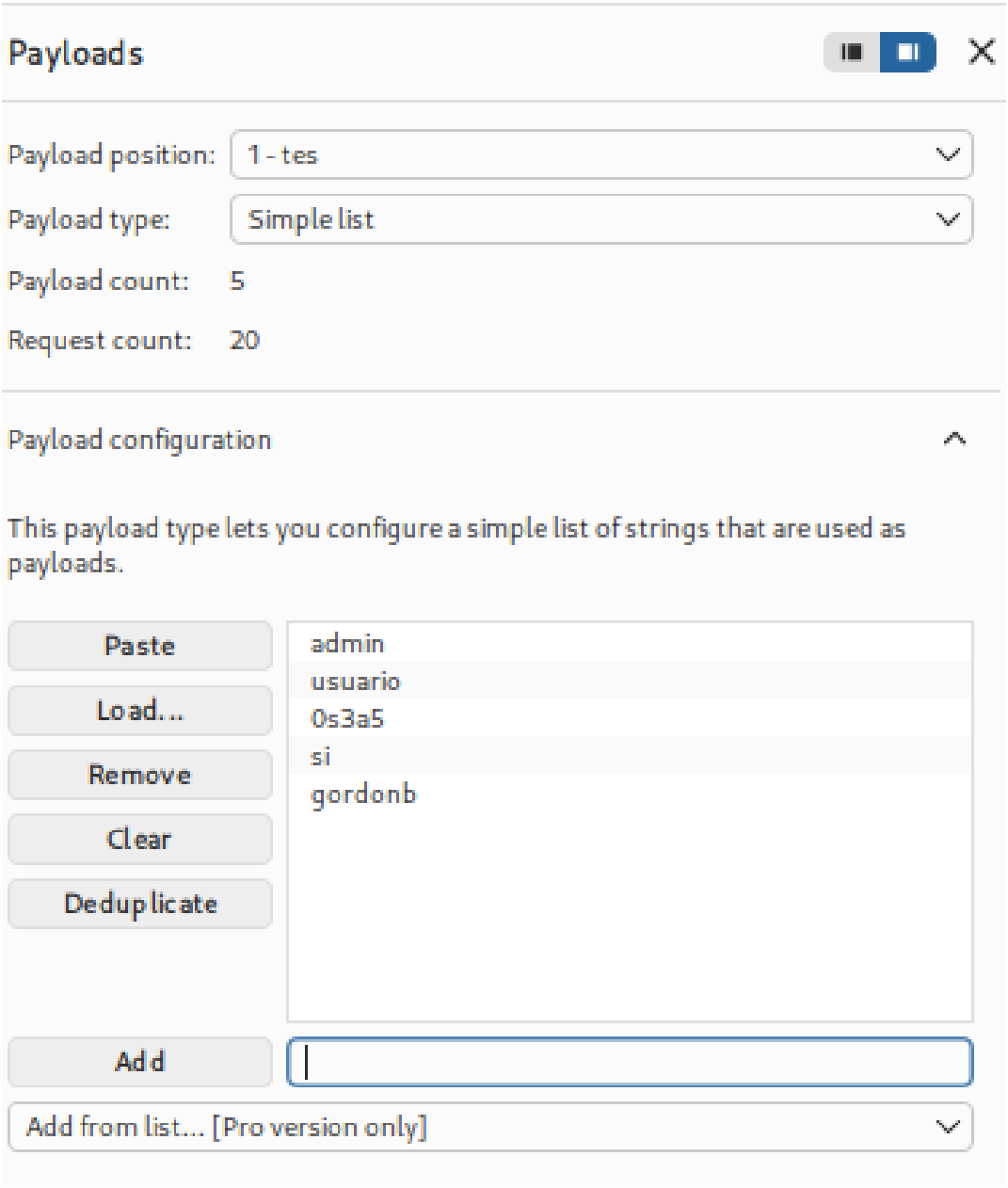


Figura 5: lista de claves utilizadas

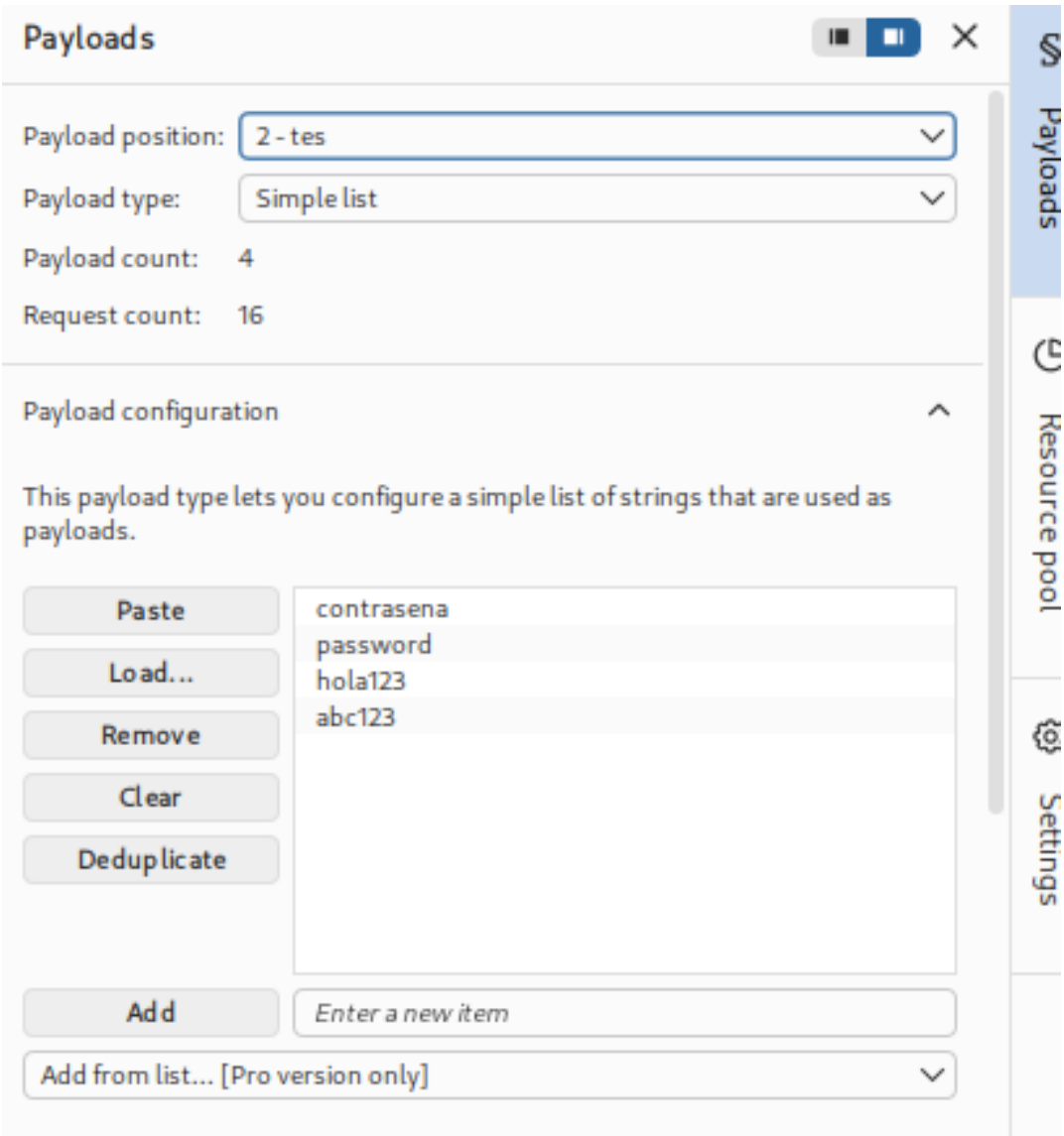
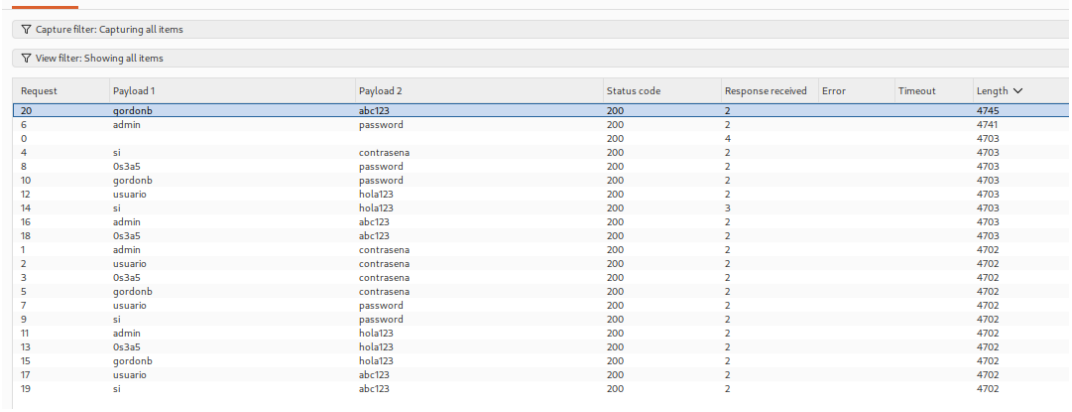


Figura 6: lista de claves utilizadas

Se evidencia en las imágenes que se muestran se añaden usuarios y contraseñas tanto por defecto como las que son de inicio de sesión correcto, esto se realiza de forma manual ya que en kali no venían las librerías de usuarios y contraseñas predefinidas

2.6. Obtención de al menos 2 pares (burp)

Al finalizar el ataque, los resultados se ordenaron por la columna *Length*: los pares correctos se distinguen por presentar una longitud de respuesta distinta al resto, ya que el servidor retorna el mensaje de bienvenida en lugar del mensaje de error.



Capture filter: Capturing all items

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length
20	gordonb	abc123	200	2			4745
6	admin	password	200	2			4741
0			200	4			4703
4	si	contrasena	200	2			4703
8	0s3a5	password	200	2			4703
10	gordonb	password	200	2			4703
12	usuario	hola123	200	2			4703
14	si	hola123	200	3			4703
16	admin	abc123	200	2			4703
18	0s3a5	abc123	200	2			4703
1	admin	contrasena	200	2			4702
2	usuario	contrasena	200	2			4702
3	0s3a5	contrasena	200	2			4702
5	gordonb	contrasena	200	2			4702
7	usuario	password	200	2			4702
9	si	password	200	2			4702
11	admin	hola123	200	2			4702
13	0s3a5	hola123	200	2			4702
15	gordonb	hola123	200	2			4702
17	usuario	abc123	200	2			4702
19	si	abc123	200	2			4702

Figura 7: Resultados del ataque Intruder ordenados por longitud

Para este caso se utilizaron 2 pares que se sabían que eran respuestas y estos son: gordonb / abc123
admin / password

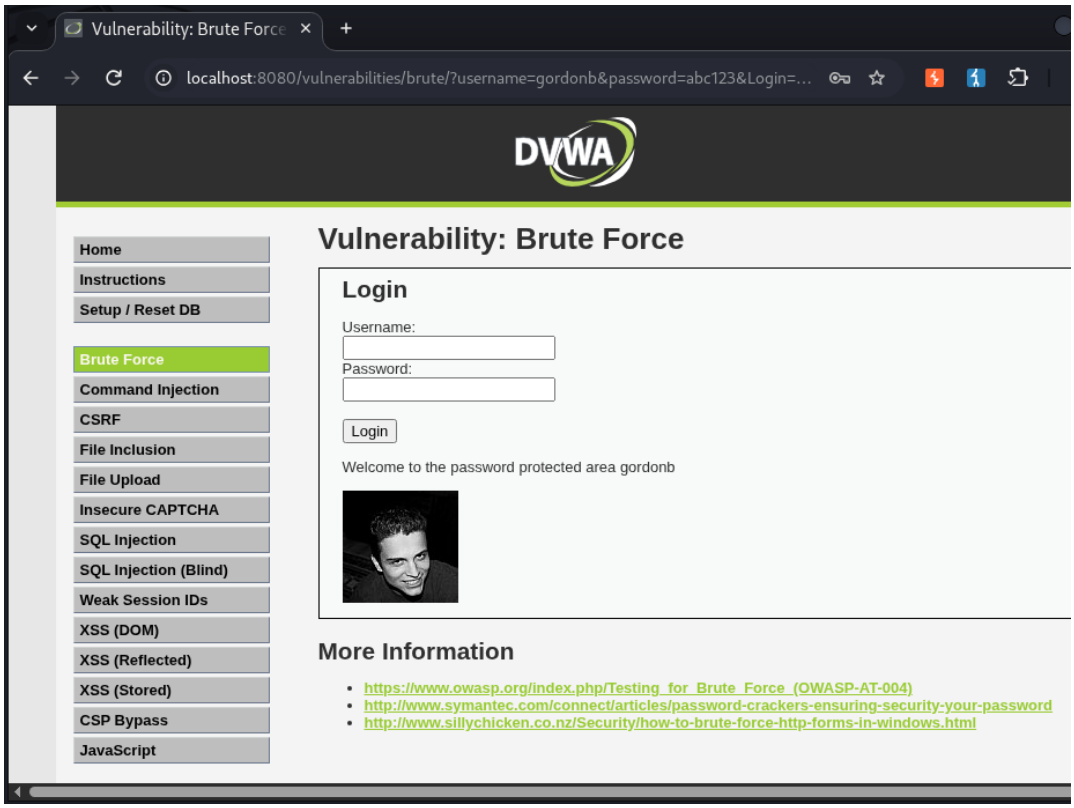


Figura 8: Ingreso por gordon

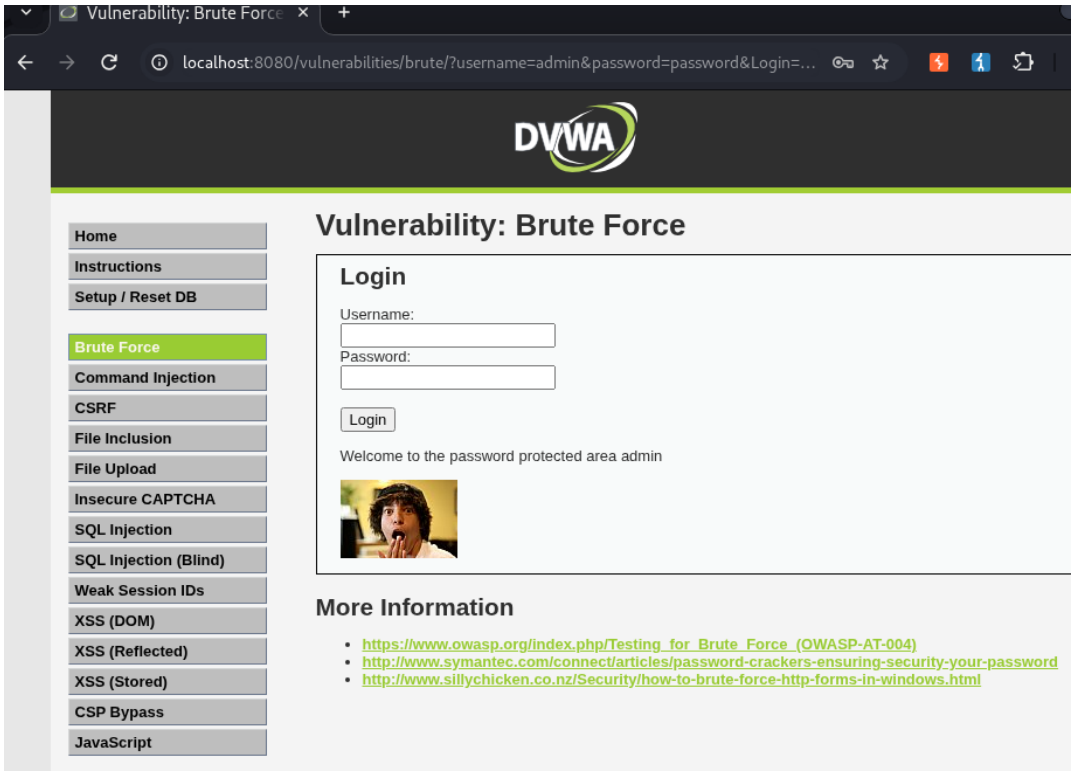


Figura 9: Ingreso por admin

2.7. Obtención de código de inspect element (curl)

Con el navegador ya logueado en DVWA, se abrió *Inspect Element* → *Network* y se recargó el formulario de `vulnerabilities/brute/` para observar la petición exacta generada por el navegador (parámetros, cabeceras y cookie de sesión).

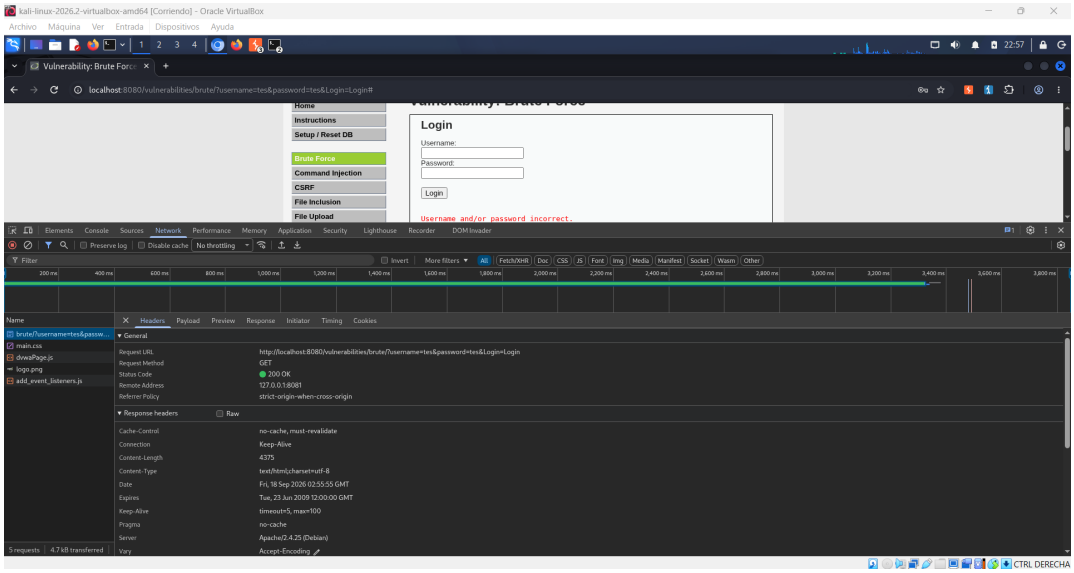


Figura 10: Herramientas de desarrolador en formulario

2.8. Utilización de curl por terminal (curl)

A partir del código obtenido en el paso anterior, se replicó la petición usando cURL, tanto para un acceso inválido como para uno válido:

```
# Acceso invalido
curl -s -i "http://localhost:8080/vulnerabilities/brute/?username=test&password=test&L
  -H "Cookie: PHPSESSID=iivpsctn4r446m34bpqt0mkg42; security=low" -o invalido.html

# Acceso valido
curl -s -i "http://localhost:8080/vulnerabilities/brute/?username=admin&password=passw
  -H "Cookie: PHPSESSID=iivpsctn4r446m34bpqt0mkg42; security=low" -o valido.html
```



Figura 11: Terminal ejecutando los comandos cURL para valido

```
┌──(kali㉿kali)-[~]
└─$ curl -i -s "http://localhost:8080/vulnerabilities/brute/?username=test&pa
ssword=test&Login=Login" \
  -H "Cookie: PHPSESSID=iivpsctn4r446m34bpqt0mkg42; security=low" > invalido.
html
```

Figura 12: Terminal ejecutando los comandos cURL para inválido

2.9. Demuestra 4 diferencias (curl)

Para la identificación de las diferencias se utilizo el comando

`diff - u valido.html invalido.html`

en donde se encontraron las siguietnes diferencias

```
┌──(kali㉿kali)-[~]
└─$ diff -u  valido.html invalido.html
── valido.html 2026-09-20 18:42:20.180628088 -0400
+++ invalido.html       2026-09-20 18:42:28.360716077 -0400
@@ -1,11 +1,11 @@
 HTTP/1.1 200 OK
-Date: Sun, 20 Sep 2026 22:42:20 GMT
+Date: Sun, 20 Sep 2026 22:42:28 GMT
 Server: Apache/2.4.25 (Debian)
 Expires: Tue, 23 Jun 2009 12:00:00 GMT
 Cache-Control: no-cache, must-revalidate
 Pragma: no-cache
 Vary: Accept-Encoding
-Content-Length: 4413
+Content-Length: 4375
 Content-Type: text/html;charset=utf-8


@@ -82,7 +82,7 @@
                <input type="submit" value="Login" name="Login">

                </form>
-               <p>Welcome to the password protected area admin</p><img src="
/hackable/users/admin.jpg" />
+               <pre><br />Username and/or password incorrect.</pre>
        </div>
```

Figura 13: Termianl con diferencias

1. **Mensaje devuelto:** la respuesta inválida muestra *"Username and/or password incorrect"*, mientras que la válida muestra *"Welcome to the password protected area"*.

2. **Longitud de la respuesta:** el tamaño en bytes del HTML retornado difiere entre ambos casos. En un caso es de 4413 y en el otro es de 4375

3. **Presencia de contenido adicional:** solo la página de acceso válido incluye el bloque de contenido correspondiente al área protegida.

4. **Clase/elemento CSS del mensaje:** DVWA marca el mensaje de error con una clase distinta (por ejemplo un `div` de alerta) que no aparece en la respuesta exitosa.

2.10. Instalación y versión a utilizar (hydra)

Para el caso de kali, Hydra ya viene instalado por lo cual solo se verifica la versión.

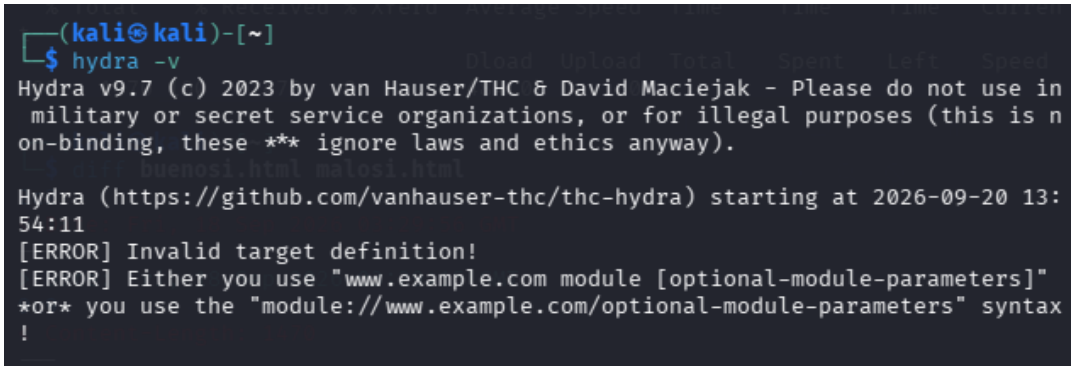


Figura 14: Version de Hydra

2.11. Explicación de comando a utilizar (hydra)

```
hydra -L users.txt -P pass.txt localhost -s 8080 http-get-form \
  "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:\
H=Cookie: PHPSESSID=iivpsctn4r446m34bpqt0mkg42; security=low:\
F=Username and/or password incorrect"
```

Para este caso se utiliza Hydra para probar diferentes contraseñas, se utilizan los campos a modificar igual que en el caso de Burpsuit y se le entrega los mismos datos que con curl

2.12. Obtención de al menos 2 pares (hydra)

En este caso se utilizaron todas las combinaciones de mas de un par de claves para entrar, en este caso hicieron 25 posibles combinaciones de las cuales solo 5 son correctas, de los cuales se obtienen las sigueinetes marcadas en color verde:

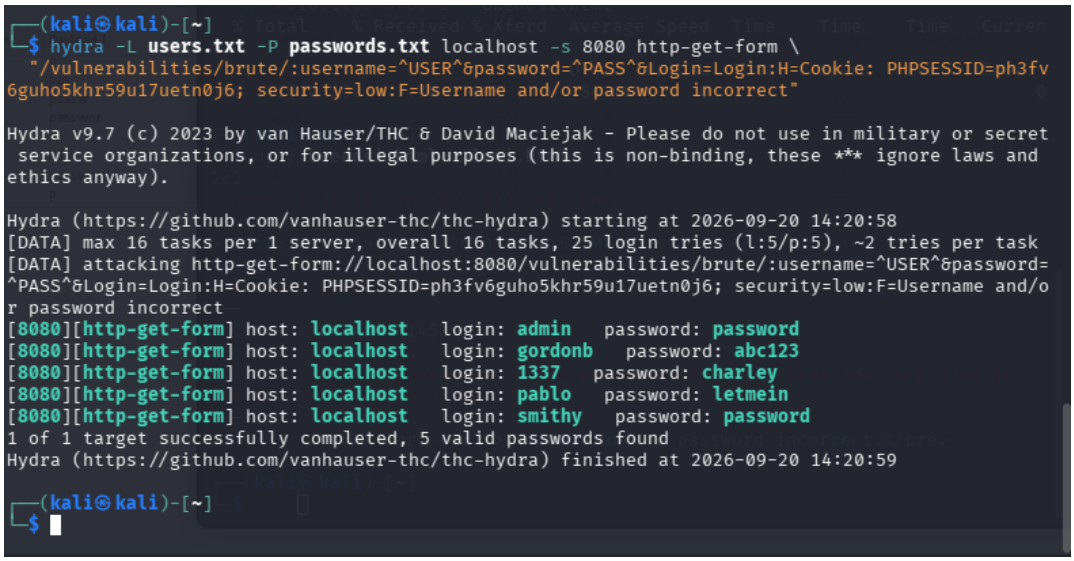


Figura 15: Terminal mostrando los pares encontrados por Hydra

Pares:
admin / password
gordonb / abc123
1337 / charley pablo / letmein
smithy / password

2.13. Explicación paquete curl (tráfico)

Analizando en Wireshark la captura del paquete GET generado por cURL (filtro `http.request.method == "GET" and http.user_agent contains "curl"`):

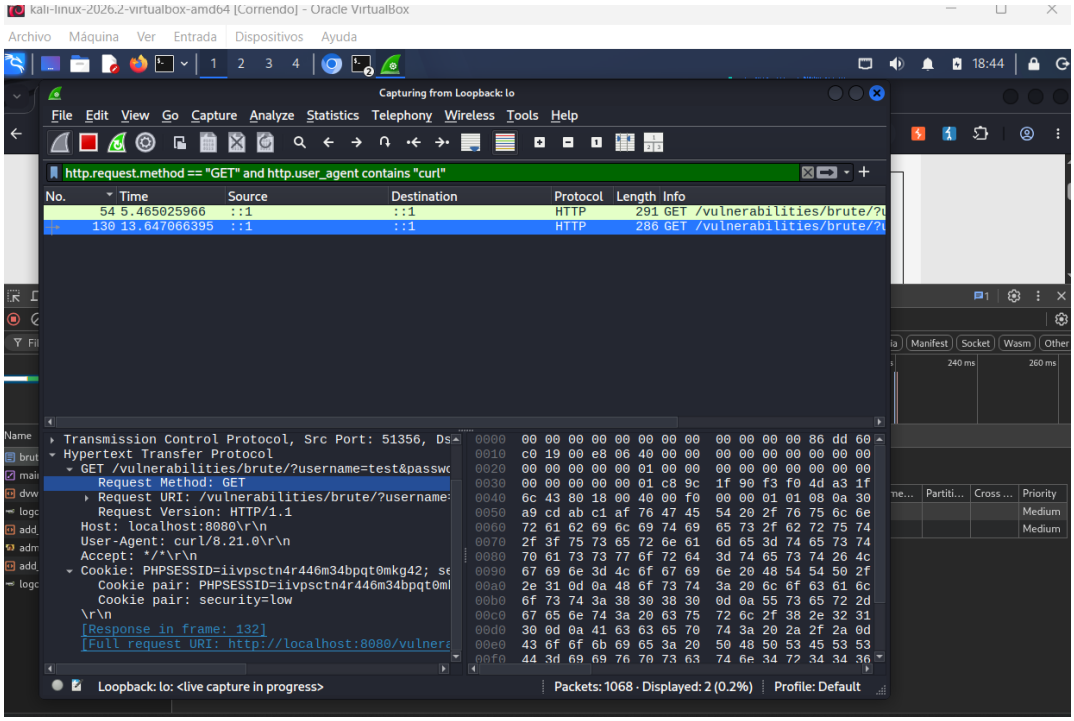


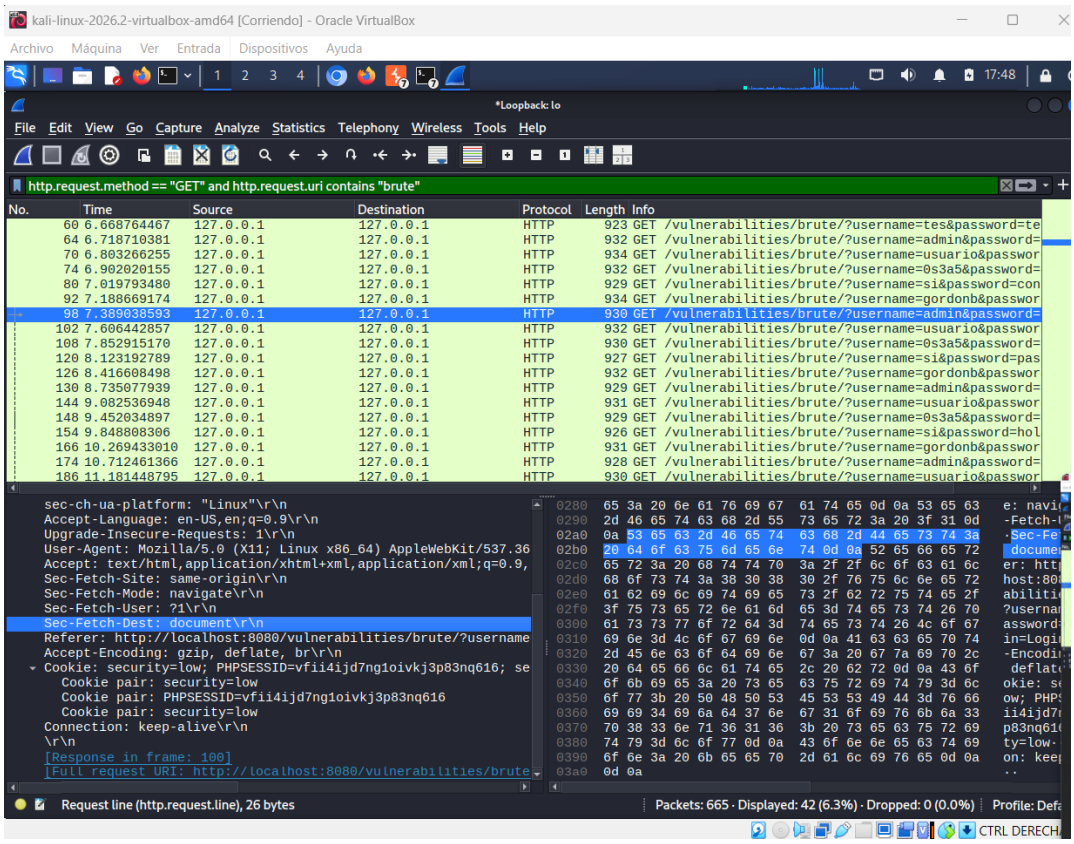
Figura 16: Paquetes de las solicitudes generadas por cURL

Del total de 1068 paquetes capturados en la interfaz de loopback, únicamente 2 corresponden a cURL (0.2 % del tráfico), en los instantes $t = 5{,}465026\,s$ y $t = 13{,}647066\,s$, es decir, separados por más de 8 segundos entre sí. Esto confirma que se trata de dos invocaciones manuales y puntuales (el acceso inválido y el válido) y no de un proceso automatizado.

- **Cabecera User-Agent:** identifica de inmediato la herramienta y hasta su versión exacta, `curl/8.21.0`.
- **Estructura minimalista:** solo incluye las cabeceras indispensables (`Host`, `User-Agent`, `Accept: */*`) además de la `Cookie` inyectada manualmente con `PHPSESSID` y `security=low`.
- **Tamaño de trama reducido:** las tramas capturadas pesan 291 y 286 bytes respectivamente, muy por debajo de los 920 bytes que genera una petición equivalente desde el navegador.
- **Direccionamiento de red:** ambas peticiones viajan sobre `::1` (loopback IPv6), a diferencia de Hydra y del navegador, que en las capturas siguientes usan `127.0.0.1` (loopback IPv4).
- **Comportamiento TCP:** cada acceso corresponde a una conexión TCP independiente y puntual donde no se ve concurrencia.

2.14. Explicación paquete burp (tráfico)

Analizando las solicitudes del ataque de Intruder (filtro http.request.method ==”GET” and http.request.uri contains ”brute”):



de las solicitudes generadas por Burp

Se identificaron 42 paquetes coincidentes de un total de 665 (6.3 %). A diferencia de Hydra, los intervalos entre solicitudes no son constantes: comienzan cerca de los 50 ms (por ejemplo, entre los tiempos 6.668764 s y 6.718701 s) y se van ampliando progresivamente hasta acercarse a los 470 ms hacia el final de la captura (entre 10.712461 s y 11.181449 s), esto porque estamos en la versión gratuita,

- **Cabecera User-Agent:** corresponde al motor Chromium del navegador embebido de Burp (`Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36...`), todo desde el navegador chorium.
- **Cabeceras Fetch/Client Hints:** las cabeceras que se encuentran aquí son correspondientes a Burp, estas son de trafico real y no se encuentran en Hydra ni tampoco en curl.
- **Cabecera Referer:** cada petición incluye el `Referer` apuntando a la petición anterior del propio formulario, nuevmanete esto no se encuentra ni en Hydra ni en curl.

- **Conexión persistente:** `Connection: keep-alive`, esto es porque es solamente una seison en la cual se ejecuto y no en casos puntuales de conexión.
- **Transmisión de credenciales:** viajan en texto claro dentro de los parámetros de la URL (método GET), sin cifrado TLS, en tramas de 923 a 934 bytes.

2.15. Explicación paquete hydra (tráfico)

Analizando las solicitudes generadas por Hydra
(filtro `http.request.method == "GET" and http.user_agent contains "Hydra"`):

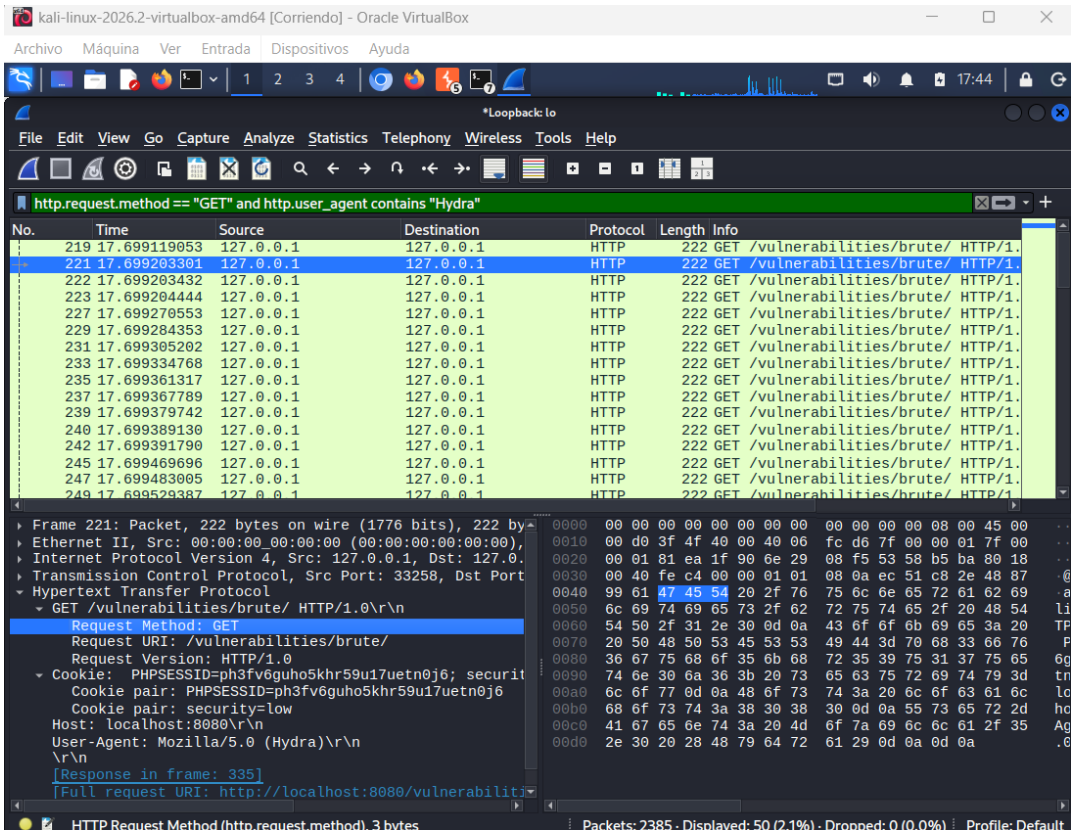


Figura 17: Paquetes de las solicitudes generadas por Hydra

Se muestran 50 paquetes coincidentes de un total de 2385 capturados (2.1 %), volumen muy superior al generado por Burp para un ataque de dimensiones similares. El tiempo entre tramas consecutivas es extremadamente reducido: por ejemplo, entre los paquetes 219 y 221 la diferencia es de apenas $84\,\mu s$ (0,000084 s), y entre los paquetes 221 y 222 de solo $131\,ns$ (0,000000131 s), todos los intervalos observados se mantienen muy por debajo del milisegundo.

- **Versión de protocolo HTTP/1.0:** a diferencia de cURL y del navegador (Burp),

que utilizan HTTP/1.1, Hydra emite sus peticiones en `HTTP/1.0`, un rasgo adicional y muy distintivo para identificarla en el tráfico.

- **Ausencia total de metadatos de navegador:** solo incluye `Host`, `User-Agent` y `Cookie`; no hay `Accept`.
- **Tamaño de trama constante:** todas las tramas capturadas pesan exactamente 222 bytes, lo que sugiere peticiones generadas por plantilla fija, sin variación de cabeceras entre intentos.
- **Concurrencia:** múltiples conexiones TCP paralelas desde distintos puertos en los cuales por el tiempo se puede ver que hay varios hilos simulataneamente funcionando.

2.16. Mención de las diferencias (tráfico)

Se resumen las diferencias observadas en la siguiente tabla comparativa, con datos extraídos directamente de las capturas:

Tabla 1: Comparativa del tráfico generado por cada herramienta

Criterio	**cURL**	**Burp Suite (Intruder)**	**THC-Hydra**
Cabecera User-Agent	`curl/8.21.0`	Chromium embebido (`AppleWebKit/537.36...`)	`Mozilla/5.0 (Hydra)`
Versión HTTP	HTTP/1.1	HTTP/1.1 (`keep-alive`)	HTTP/1.0
Cabeceras `Sec-Fetch-*` / Client Hints	Ausentes	Presentes y detalladas	Ausentes
Tamaño de trama	286–291 bytes	923–934 bytes	222 bytes (constante)
Cadencia entre solicitudes	Manual, ∼8 s entre accesos	Variable, 50 ms a 470 ms	Microsegundos a <1 ms
Concurrencia y sockets	1 socket puntual por acceso	1 socket persistente	Múltiples sockets TCP en paralelo
Direccionamiento	`::1` (IPv6 loopback)	`127.0.0.1` (IPv4)	`127.0.0.1` (IPv4)
Riqueza de cabeceras	Mínima indispensable	Completa (idioma, encoding, UI, Referer)	Sintética y reducida

2.17. Detección de SW (tráfico)

Un sistema de detección de intrusos (IDS), un firewall de aplicaciones web (WAF) o un analizador de red puede identificar la herramienta empleada a través de cuatro mecanismos principales:

1. **Inspección de firmas en el User-Agent:** reglas estáticas detectan de inmediato subcadenas como `Hydra` o `curl`, presentes por defecto en la cabecera de capa de aplicación.

2. **Fingerprinting heurístico de cabeceras:** los navegadores reales respetan un orden canónico de cabeceras y transmiten directivas de contexto (*Sec-Fetch-**, *Accept-Language*, *Referer*, etc.). Las herramientas de scripting o consola omiten estas cabeceras, lo que permite a un WAF clasificarlas como tráfico sintético.

3. **Versión de protocolo HTTP:** el uso de HTTP/1.0 por parte de Hydra, en comparación al HTTP/1.1 con conexiones persistentes que usan cURL y los navegadores, es en sí mismo un indicador adicional de automatización.

4. **Análisis de flujo y frecuencia temporal:** Hydra satura la conexión mediante ráfagas de solicitudes con deltas de microsegundos, activando alertas volumétricas; Burp, en cambio, genera intervalos variables de decenas a cientos de milisegundos (por ser la versión que no se paga), y cURL genera accesos puntuales y espaciados en segundos — ambos patrones, aunque distintos entre sí, resultan incompatibles con el ritmo de tecleo de un usuario humano.

Conclusiones y comentarios

Al finalizar el laboratorio se puede evidenciar porque no se deben dar laboratorios para fiestas patrias(hay mas café y empanadas en mi sistema que ganas de hacer esto). Pero si se logro identificar los distintos tipos de ataque basados en fuerza bruta y su impacto en la seguridad de las paginas web. Se logro evidenciar por medio de wireshrak que cada tipo de ataque ya sea curl burp o Hydra dejan un rastro o huella diferente e identificable en la red al realizar los ataques, esto nos deja como conclusión de que no sirve de nada tener la mejor contraseña o similar si no se tiene algun mecanismo de defensa en contra de estos tipos de ataque o de detección de patrones, ya que se pudo ver que hay algunos métodos que inyectan intentos cada mili segundos, y esto puede ser un aviso de vulnerabilidad que debe levantar alertas.